Demo 1: Data Analysis Agent

This is a Data Analysis application built using Phidata's agent framework that allows users to upload datasets and receive structured, AI-powered summaries. The application automatically reads files, analyzes key statistics, chunks metadata, embeds it using Sentence Transformers, and stores it in Pinecone for semantic retrieval. It features a conversational agent interface powered by Google Gemini and is deployed with a user-friendly UI via Streamlit.

Scenario and Problem Statement

- **Scenario**: Data analysts and other professionals often work with large numbers of raw tabular data files that lack proper documentation or summaries. Manually inspecting each file is repetitive, error-prone, and time-consuming. There is a need for a smart assistant to automatically analyze, summarize, and remember key features of these files for future reference.
- **Problem Statement**: The goal is to build an intelligent Data Analysis Agent to help users efficiently understand and organize datasets. The solution should allow file uploads, automatically generate statistical and structural summaries, and embed this information into a vector database for semantic similarity searches. The system should enable users to retrieve datasets using natural language queries and provide conversational interactions.

Solution Approach

The application uses a modular, agent-based architecture built with Phidata, Google Gemini, Sentence Transformers, Pinecone, and Streamlit. The main goal is to transform raw CSV files into analyzable, semantically searchable knowledge accessible through a conversational AI web interface.

1. Environment Setup and API Configuration

The system loads API keys for Google Gemini and Pinecone, which are stored in a `.env` file or as environment variables.

2. Model Initialization

Two core models are initialized:

- **Gemini (LLM)**: The main reasoning engine for interpreting queries and generating responses.

- **Sentence Transformer (`all-MiniLM-L6-v2`)**: Converts text summaries into vector embeddings for semantic tasks.

3. Vector Database Setup (Pinecone)

Pinecone is used to store and search CSV metadata. The setup creates an index named `data-insights` if it doesn't exist and handles errors gracefully.

4. Agent Tools

Three key functions are implemented as tools for the Phidata Agent:

- `describe_data(filepath: str) -> str`: Reads a CSV file and generates a summary using `pandas.DataFrame.describe()`.
- `embed_and_store(filepath: str) -> str`: Converts the summary into chunks of about 300 characters, embeds them, and stores the vectors in Pinecone.
- `search_similar(query: str) -> str`: Encodes a natural language query and searches Pinecone for similar metadata chunks, returning IDs and similarity scores.

5. Agent and Workflow Construction

A Phidata Agent encapsulates these tools to support conversational queries like "Describe the uploaded CSV". A Workflow wraps the agent to allow for future extensibility, such as adding data cleaning tasks.

6. Front-End User Interface (Streamlit)

A Streamlit front-end supports:

- **File Upload**: Users can upload a `.csv` file.
- **Automated Summary**: The agent automatically generates and displays a statistical summary upon upload.
- **Similarity Search**: Users can find similar datasets using natural language queries.
- **CSV Download**: The generated summary can be downloaded.

7. Resilience and Error Handling

Fault tolerance is managed by wrapping index operations in `try-except` blocks and showing error messages in the UI if external services encounter issues.

Key Technologies Used

Component	Purpose
Phidata Agent	Manages tool orchestration and workflows
Google Gemini	Generates summaries and interprets queries
Sentence Transformers	Embeds dataset metadata
Pinecone	Stores and retrieves semantic vector data
Streamlit	Provides a user-facing web interface
Pandas	Handles CSV parsing and analysis

Export to Sheets

Project and Solution Structure

- **Project Structure**:
    1. `app4.py`: Main script with core logic.
    2. `data.csv`: Dataset with employee records.
    3. `requirements.txt`: Lists project dependencies.
    4. `.env`: Stores environment variables.
- **Solution Structure**:
    1. **Upload Interface**: Streamlit for CSV uploads.
    2. **Statistical Description Tool**: Generates summaries using `pandas.DataFrame.describe()`.
    3. **Chunking & Embedding Tool**: Chunks and embeds summaries.
    4. **Vector Store Integration**: Pinecone stores embeddings for similarity search.
    5. **Agent with Tools**: Orchestrates tools based on user prompts.
    6. **UI Output**: Streamlit displays summaries and allows downloads.

Stepwise Solution and Code

Step 1: Setting up the Environment

Initialize environment variables for API keys.

Python

```
None
os.environ['GOOGLE_API_KEY'] = 'your-google-key'
os.environ['PINECONE_API_KEY'] = 'your-pinecone-key'
```

Step 2: Uploading and Ingesting CSV

Use Streamlit's file uploader to read a CSV file with Pandas.

Python

```
None
uploaded_file = st.file_uploader("Upload a CSV", type=["csv"])
df = pd.read_csv(uploaded_file)
```

Step 3: Generating Descriptive Stats

Create a function to describe the data in a file.

Python

```
None
def describe_data(filepath: str) -> str:
    df = pd.read_csv(filepath)
    return df.describe().to_string()
```

Step 4: Chunking and Embedding Metadata

Split the summary into smaller chunks and create embeddings.

Python

```
None
chunks = [summary[i:i+300] for i in range(0, len(summary), 300)]
embedding = embedding_model.encode(chunk).tolist()
```

Step 5 & 6: Storing and Querying in Pinecone

Store the embeddings in a Pinecone index and enable semantic search.

Python

```
None
pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
pc.create_index(name="data-insights", dimension=384)
index.upsert(vectors)
```

Step 7: Assembling the Agent

Integrate all tools using Phidata's Agent and Workflow.

Python

```
None
agent = Agent(tools=[describe_data, embed_and_store,
search_similar], model=model)
workflow = Workflow(agents=[agent])
```

Step 8: Building the UI with Streamlit

Create a Streamlit interface for user interaction.

Python

```
None
st.title("CSV Data Insights")
st.text(agent.run("Describe the dataset in uploaded_data.csv"))
```

User Interface and Output

- **User Interface**: A web page titled "Data Analysis Agent" with a drag-and-drop area to upload a CSV file.
- **Output**: A table displaying the statistical summary of the uploaded dataset, including count, mean, standard deviation, min, max, and quartile values for each numerical column.

Conclusion

This solution demonstrates how to combine Phidata, Google Gemini, Pinecone, and Sentence Transformers to create a powerful, AI-first data exploration tool. The agent assists users in summarizing, storing, and retrieving insights from files through a conversational Streamlit web app